

UAH Mars Drone ASW

Architecture Design Document

Index

- 1 Introduction
- 2 Applicable and Reference Documents
- 3 Terms, definitions and abbreviated terms
- 4 Software Design Overview
- 5 Software Design
 - 5.1 Software Static Architecture
 - 5.2 Software Dynamic Architecture
 - 5.2.1 Computational Model
 - 5.2.2 Software Behaviour
 - 5.3 Real-Time Requirements
 - 5.4 Software Components Design
 - 5.5 SRD Req Coverage Matrix
- 6 Traceability

5 Software Design

5.1 Software Static Architecture

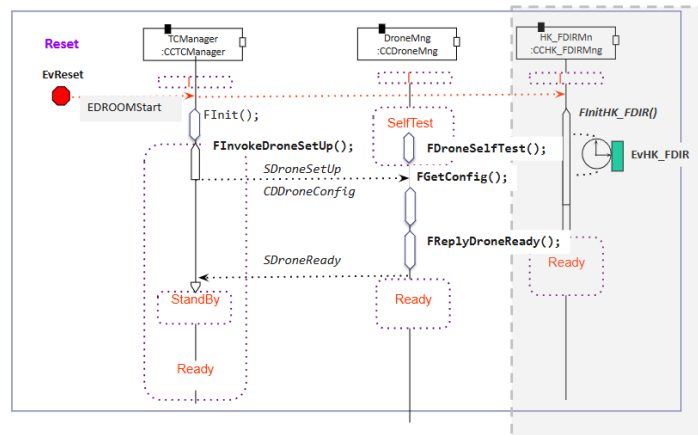
5.2 Software Dynamic Architecture

5.2.1 Computational Model

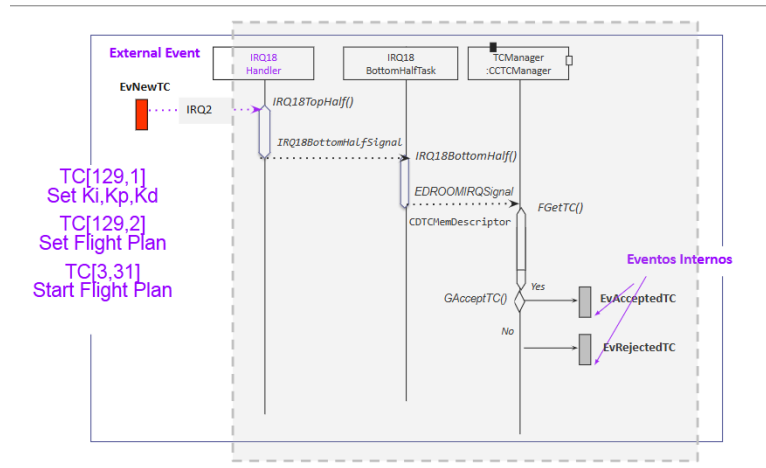
5.2.2 Software Behaviour

Event	Description
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptation
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
EvDroneTC (internal)	Inicializes drone
EvInitFlightPlan (internal)	Starts Flight plan
EvFlightCtrl (timing)	Control during flight plan
EvFlightPlaneDone (internal)	Ends flight plan

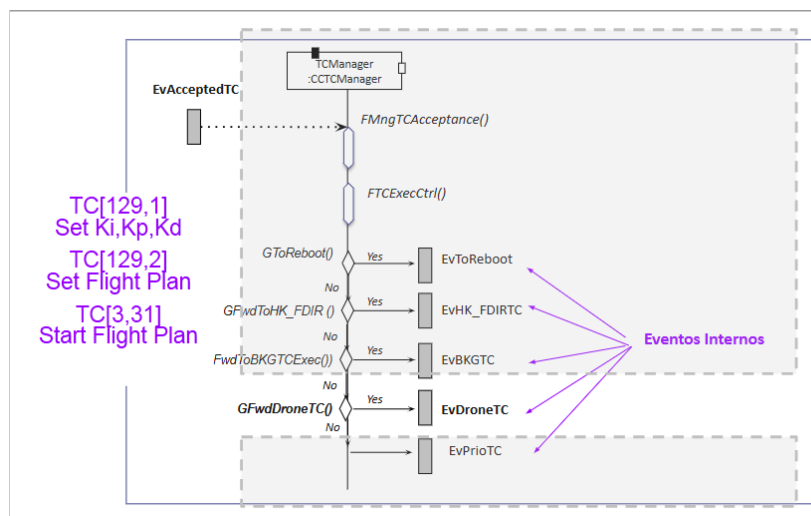
EvReset



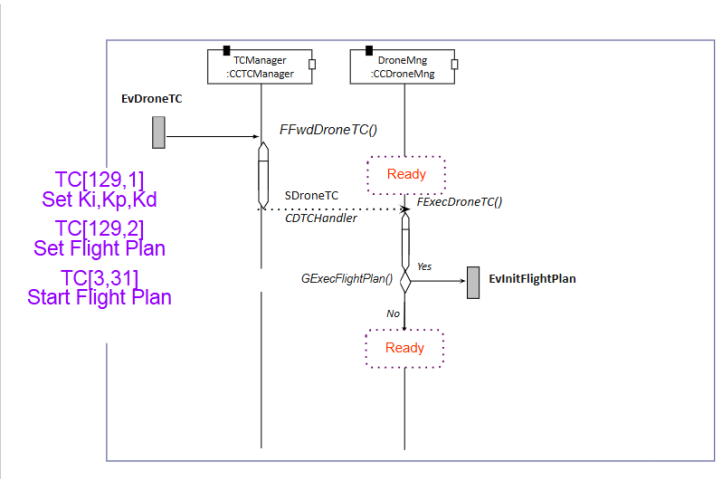
EvNewTC



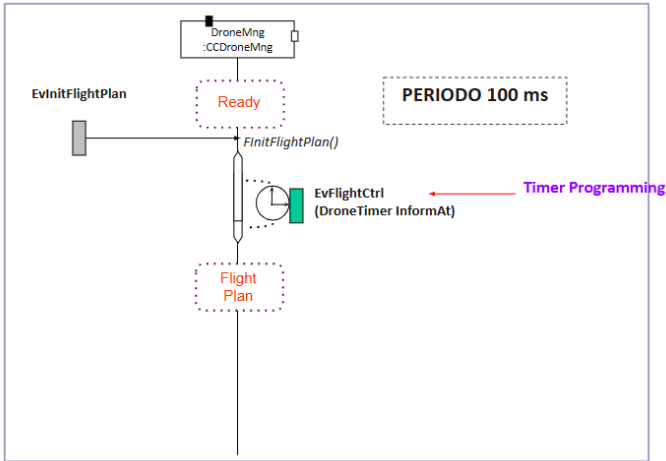
EvAcceptedTC



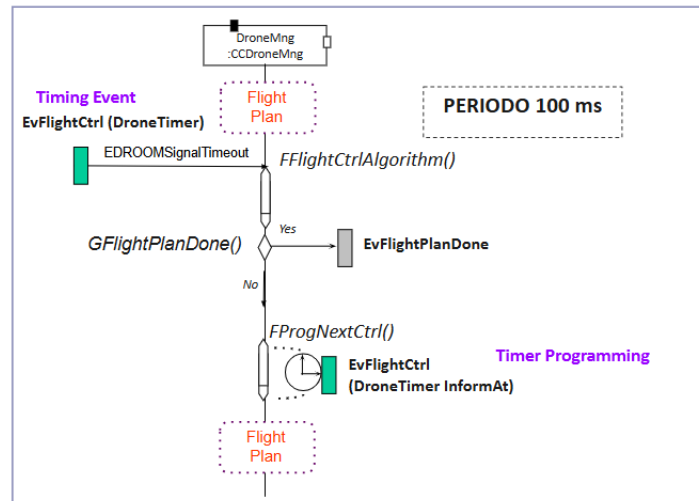
EvDroneTC



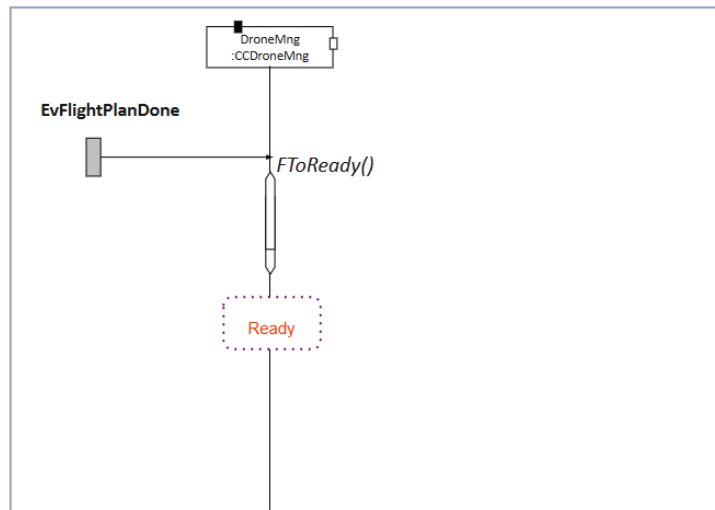
EvInitFlightPlan



EvFlightCtrl



EvFlightPlanDone



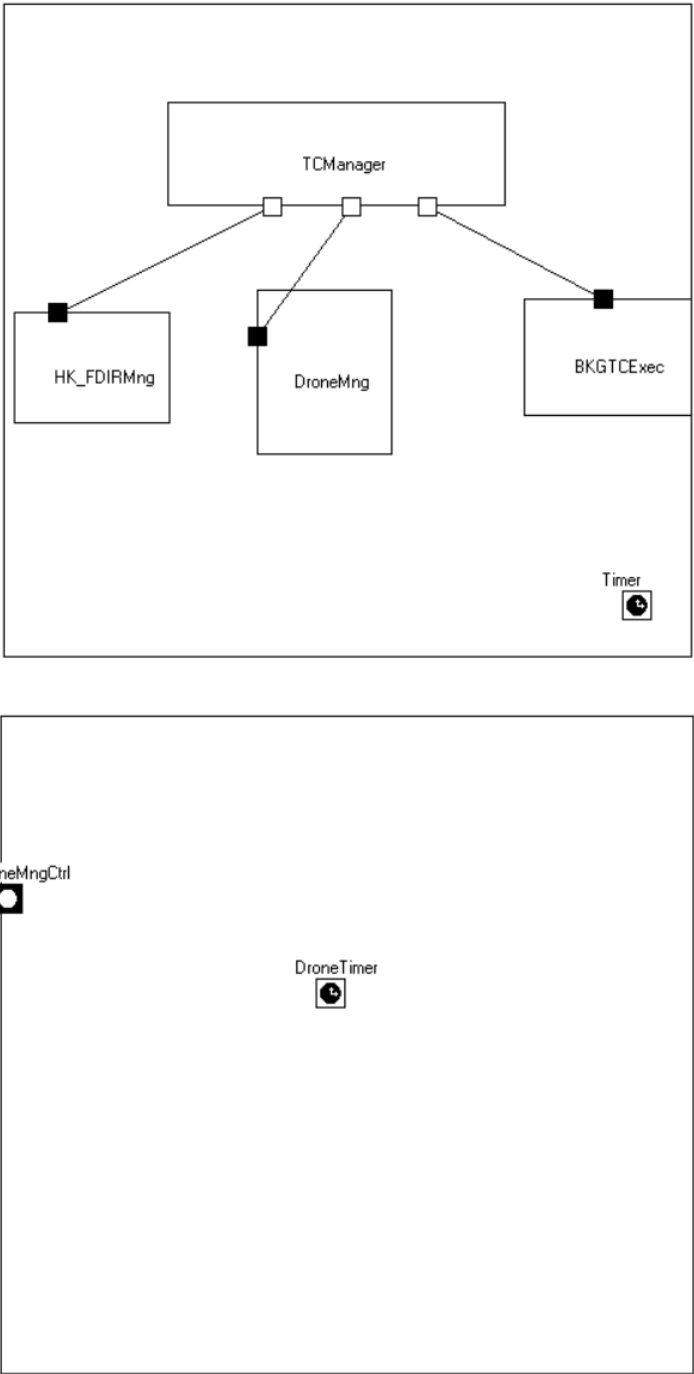
5.3 Real-Time Requirements

5.4 Software Components Design

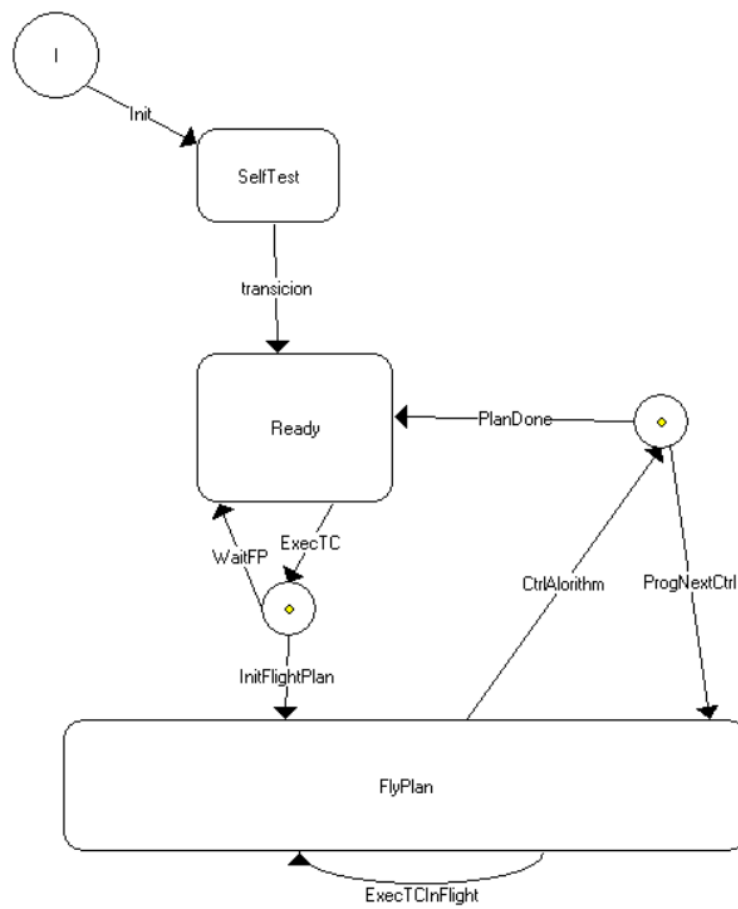
5.4.1 CCDroneMng

5.4.1.1 Interfaces

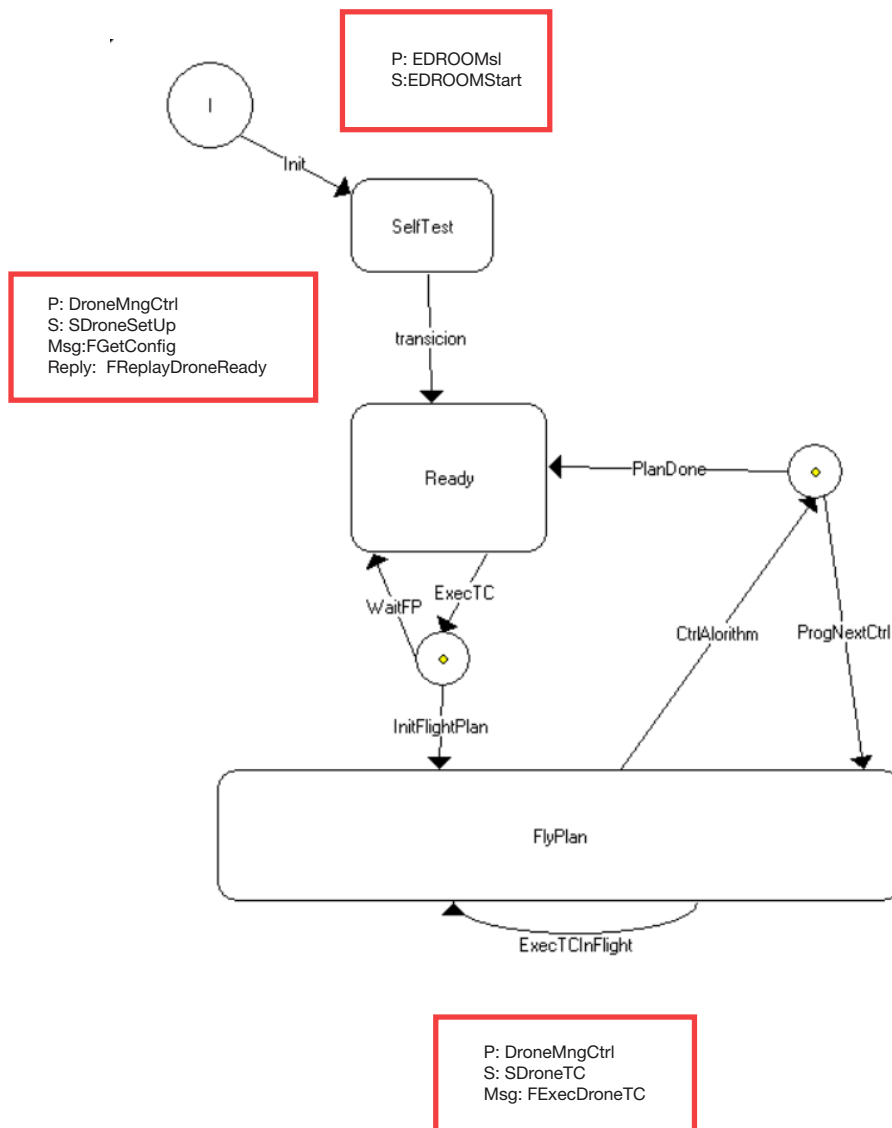
CCDroneMng Interfaces

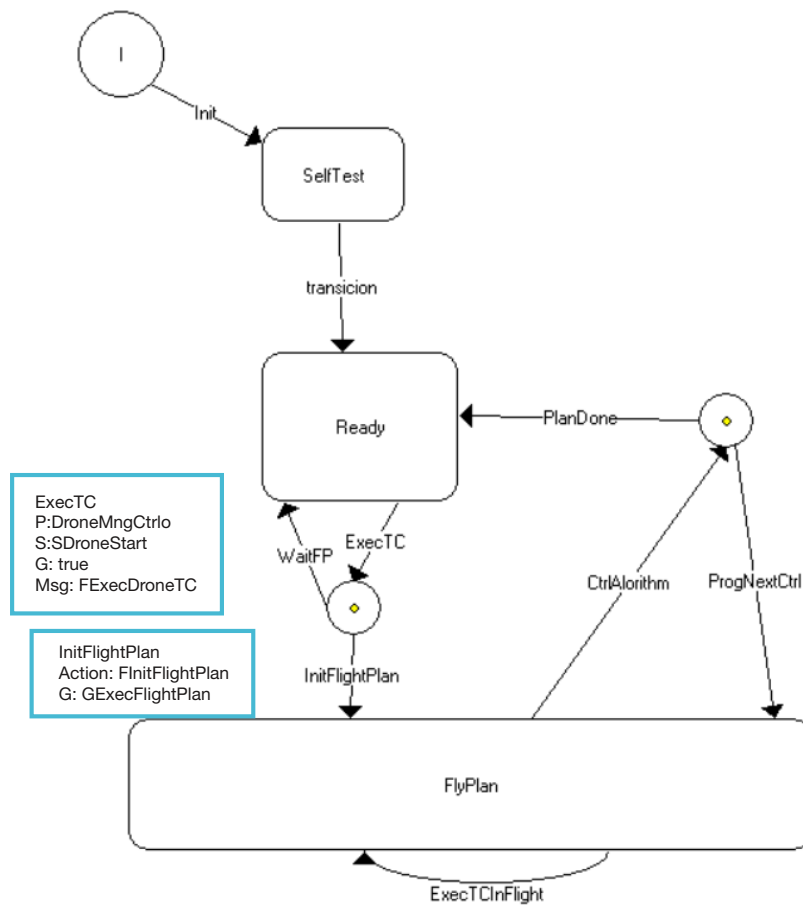


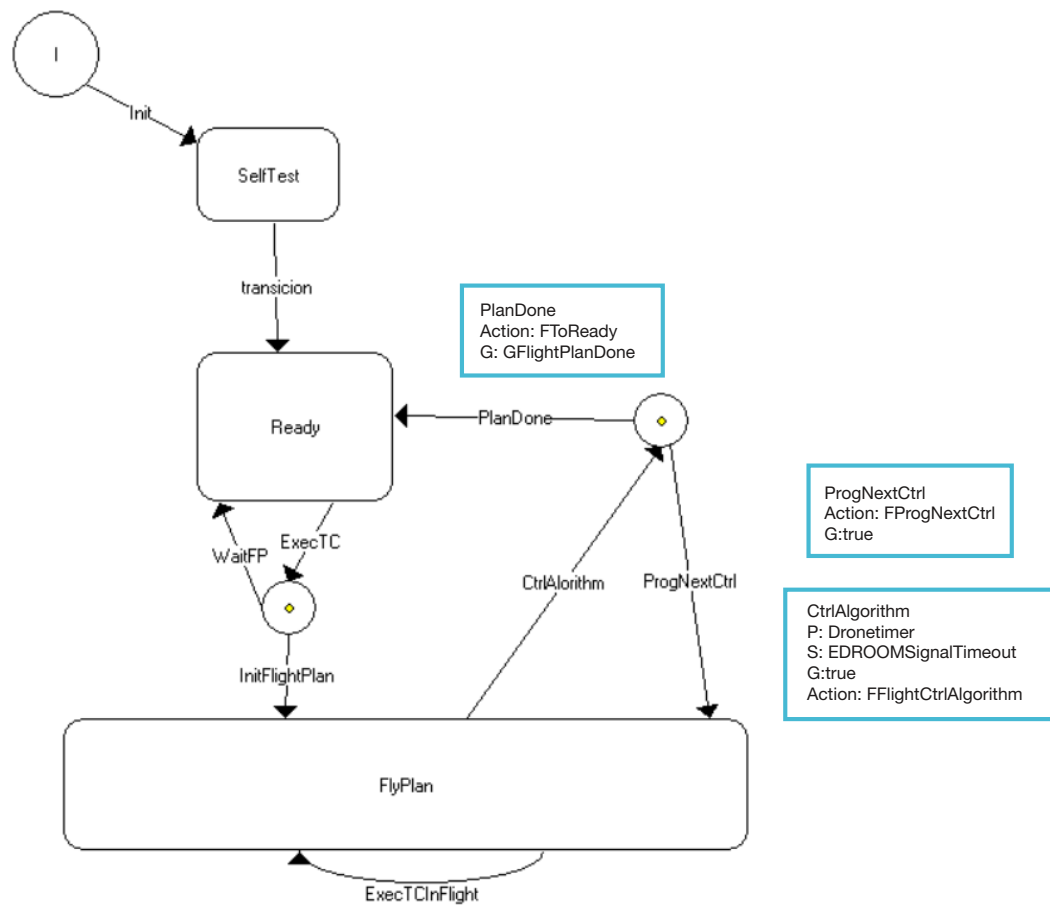
5.4.1.1 Behaviour



Vars:
Pr_Time VNextCtrl;







Declaration: `FGetConfig()`

Brief

```
{  
    CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *)Msg->data;  
  
    // Data access  
    // ... =varSDroneSetUp;  
pus_servicel29_setup(varSDroneSetUp);  
}
```

Declaration: `FReplayDroneReady()`

Brief

```
{  
  
pus_servicel29_drone_ready();  
  
    Msg->reply(SDroneReady);  
}
```

Declaration: `FExecDroneTC()`

Brief

```
{
    CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;

    // Data access
    // ... =varSDroneTC;

    // Data access
    // ... =varSDroneTC;

    varSDroneTC.ExecDroneTC();
}
```

Declaration: `FInitFlightPlan()`

Brief

```
{
    Pr_Time time;

    //Timing Service useful methods

    //time.GetTime(); // Get current monotonic time
    //time.Add(X,Y); // Add X sec + Y microsec

    time.GetTime(); // Get current monotonic time
    time+=Pr_Time(0,100000); // Add X sec + Y microsec
    VNextCtrl=time;
    pus_servicel29_init_flight_plan();

    DroneTimer.InformAt( time );
}
```

Declaration: `FFlightCtrlAlgorithm()`

Brief

```
pus_servicel29_flight_ctrl_algorithm();
```

Declaration: `FProgNextCtrl()`

Brief

```
{  
    Pr_Time time;
```

```
        //Timing Service useful methods  
  
        //time.GetTime(); // Get current monotonic time  
        //time.Add(X,Y); // Add X sec + Y microsec  
VNextCtrl+= Pr_Time(0,100000); // Add X sec + Y microsec  
time=VNextCtrl;
```

```
        DroneTimer.InformAt( time );  
}
```

Declaration: `FExecDroneTC()`

Brief

```
{
    CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;

    // Data access
    // ... =varSDroneTC;

    // Data access
    // ... =varSDroneTC;

    varSDroneTC.ExecDroneTC();
}
```

Declaration: `FDroneSelfTest()`

Brief

```
pus_servicel29_drone_selftest();
```

Declaration: `FToReady()`

Brief

```
pus_servicel29_drone_ready();
```

Declaration: `GExecFlightPlan()`

Brief

```
return pus_servicel29_flight_plan_ready();
```

Declaration: `GFlightPlanDone()`

Brief

```
return pus_servicel29_flight_plan_done();
```